

Pontificia Universidad Católica de Chile

Facultad de Ingeniería

DCCUber

Inteligencia Artificial

Segundo Semestre 2026

Albert Roca PAS816129

04/10/2026

Contents

1 DCCUber: aprendizaje reforzado **2**

1.1 Uso de IA y planteamiento . . . 2

1.2 Actividad 1 — Hiperparámetros . . . 2

1.2.1 1.a — Efecto de α . . . 2

1.2.2 1.b — Efecto de γ . . . 3

1.2.3 1.c — Comparación de los agentes A y B . . . 3

1.2.4 1.d — Exploración y explotación ε-greedy . . . 3

1.2.5 1.e — Decaimiento de ε y cálculo tras 1000 episodios . . . 4

1.2.6 1.f — Relación entre α y ε . . . 5

1.3 Actividad 2 — Implementación . . . 5

1.4 Actividad 3 — Cinco experimentos . . . 6

1.4.1 Experimento 1 — Factor de descuento γ . . . 6

1.4.2 Experimento 2 — Tasa de aprendizaje α/LR . . . 7

1.4.3 Experimento 3 — Exploración . . . 10

1.4.4 Experimento 4 — Número de episodios . . . 11

1.4.5 Experimento 5 — Agente ideal . . . 13

1.5 Actividad 4 — SlimeVolleyGym . . . 15

1.5.1 Pregunta 1 — Espacios de estado . . . 15

1.5.2 Pregunta 2 — Límites de una Q-table . . . 15

1.5.3 Pregunta 3 — Uso de redes neuronales . . . 16

1.5.4 Pregunta 4 — Representación de la red . . . 16

1.5.5 Pregunta 5 — Ventaja de las redes neuronales . . . 16

1.6 Conclusiones . . . 17

1.7 Bonus: DCCampeonato. . . . 18

1.8 Anexo de IA . . . 18

1 DCCUber: aprendizaje reforzado

1.1 Uso de IA y planteamiento

Para la realización de esta parte del trabajo se utilizaron herramientas de inteligencia artificial generativa como apoyo para comprender el funcionamiento de Q-Learning, revisar la implementación, estructurar los experimentos e interpretar los resultados obtenidos. Las métricas y resultados numéricos presentados en esta sección proceden de la ejecución del código desarrollado para el entorno `Taxi-v4`.

El objetivo consiste en entrenar un agente capaz de recoger a un pasajero y llevarlo hasta su destino mediante aprendizaje reforzado. El entorno `Taxi-v4` dispone de 500 estados discretos y seis acciones posibles, por lo que el conocimiento del agente puede representarse mediante una Q-table de dimensiones:

$$500 \times 6.$$

Cada elemento $Q(s, a)$ representa el valor esperado de realizar la acción a encontrándose en el estado s.

Durante el entrenamiento se utiliza una estrategia ε-greedy, combinando exploración de nuevas acciones con explotación de las acciones que actualmente presentan mayor valor Q. Para evaluar la calidad real de la política aprendida, los agentes se evalúan posteriormente sin exploración, seleccionando siempre la acción de mayor valor.

Se considera que un episodio ha sido resuelto correctamente cuando el entorno termina mediante una entrega efectiva del pasajero. Esta evaluación se separa de la recompensa observada durante el entrenamiento, ya que esta última también incluye el coste asociado a las acciones exploratorias.

1.2 Actividad 1 — Hiperparámetros

1.2.1 1.a — Efecto de α

La tasa de aprendizaje α determina cuánto modifica el agente sus estimaciones después de observar una nueva transición. La actualización utilizada en Q-Learning es:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

El término entre corchetes representa el error temporal o *temporal difference error*.

Con un valor reducido, por ejemplo $\alpha = 0.1$, el agente incorpora únicamente una fracción de la nueva información en cada actualización. Esto produce un aprendizaje más gradual y puede requerir un mayor número de episodios.

Si se utiliza $\alpha = 1$, el valor anterior se reemplaza completamente por el nuevo objetivo temporal:

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a').$$

Esto puede acelerar el aprendizaje, especialmente en un entorno determinista, aunque también hace que cada nueva experiencia tenga una influencia mucho mayor.

Por tanto, un valor elevado de α permite modificar la Q-table rápidamente, mientras que un valor pequeño genera un aprendizaje más progresivo. La mejor elección depende del entorno y del número de episodios disponibles.

1.2.2 1.b — Efecto de γ

El factor de descuento γ determina la importancia que se concede a las recompensas futuras.

Cuando γ es cercano a cero, el agente mira las recompensas cercanas. En cambio, cuando γ se aproxima a 1, las recompensas obtenidas varios pasos más adelante continúan influyendo de forma importante en la decisión actual.

Esto resulta especialmente relevante en `Taxi-v4`. Durante el trayecto, cada movimiento supone un coste, mientras que la recompensa positiva principal aparece cuando se completa correctamente la entrega del pasajero.

Con un valor como $\gamma = 0.1$, una recompensa situada muchos pasos en el futuro pierde rápidamente influencia. Con $\gamma = 1$, en cambio, la recompensa final puede propagarse hacia decisiones tomadas mucho antes.

Por ello, en un problema de este tipo es razonable esperar que valores elevados de γ permitan aprender rutas más útiles a largo plazo.

1.2.3 1.c — Comparación de los agentes A y B

El agente A utiliza:

$$\alpha = 0.9, \qquad \gamma = 0.1,$$

mientras que el agente B utiliza:

$$\alpha = 0.2, \qquad \gamma = 0.8.$$

El agente A modificará sus valores Q rápidamente debido a su elevada tasa de aprendizaje. Sin embargo, su factor de descuento reducido hace que conceda poca importancia a recompensas futuras.

El agente B actualiza la tabla de manera más progresiva, pero tiene en cuenta en mayor medida las consecuencias futuras de sus acciones.

En el caso de Taxi, esto es especialmente importante porque una serie de movimientos con recompensa inmediata negativa puede ser necesaria para llegar finalmente al pasajero y posteriormente al destino.

Por este motivo, después de un número suficientemente elevado de episodios, esperaríamos que el agente B aprendiera una política más adecuada para completar rutas completas. El agente A puede reaccionar rápidamente ante recompensas o penalizaciones cercanas, pero tiene más dificultades para relacionar acciones tempranas con la recompensa final de la entrega.

1.2.4 1.d — Exploración y explotación ε-greedy

Durante el entrenamiento se utiliza una política ε-greedy.

Con probabilidad ε, el agente selecciona una acción aleatoria:

$$a \sim \text{acción aleatoria},$$

mientras que con probabilidad $1 - \varepsilon$ selecciona:

$$a = \arg\max_a Q(s, a).$$

La primera situación corresponde a la **exploración**, ya que permite descubrir nuevas acciones y rutas.

La segunda corresponde a la **explotación**, utilizando la mejor acción conocida hasta el momento.

Al inicio del entrenamiento resulta conveniente explorar con frecuencia porque la Q-table todavía no contiene información útil. A medida que el agente aprende, resulta razonable reducir progresivamente la exploración y explotar con mayor frecuencia la política adquirida.

1.2.5 1.e — Decaimiento de ε y cálculo tras 1000 episodios

Considerando la expresión teórica:

$$\varepsilon_n = \varepsilon_0 \cdot decay^n,$$

un valor de *decay* próximo a 1 implica que la exploración disminuye lentamente.

Si:

$$\varepsilon_0 = 1$$

y:

$$decay = 0.995,$$

después de 1000 episodios se obtiene:

$$\varepsilon_{1000} = 0.995^{1000} \approx 0.006654.$$

Por tanto, la probabilidad de seleccionar una acción aleatoria sería aproximadamente:

$$0.665\,\%.$$

En la implementación realizada se utiliza una parametrización ligeramente diferente:

$$\varepsilon = \varepsilon_{\min} + (\varepsilon_{\max} - \varepsilon_{\min})e^{-\lambda n},$$

donde λ corresponde a `epsilon_decay_rate`. En este caso, cuanto mayor es el valor de λ, más rápidamente se reduce la exploración.

1.2.6 1.f — Relación entre α y ε

Los parámetros α y ε controlan aspectos distintos del aprendizaje, pero su efecto está relacionado.

ε determina qué experiencias obtiene el agente, mientras que α determina cuánto modifica la Q-table a partir de cada una de ellas.

Por ejemplo, un agente puede explorar una gran cantidad de estados, pero si α es extremadamente pequeño necesitará muchas visitas antes de consolidar lo aprendido.

En el caso contrario, un α elevado combinado con una reducción excesivamente rápida de ε puede provocar que el agente aprenda intensamente a partir de un conjunto limitado de experiencias y deje de explorar antes de haber descubierto rutas mejores.

Por tanto, la exploración determina la diversidad de experiencias y la tasa de aprendizaje determina la velocidad con la que estas experiencias modifican la política.

1.3 Actividad 2 — Implementación

El agente utiliza una Q-table de dimensiones:

$$Q \in \mathbb{R}^{500\times 6},$$

correspondiente a los 500 estados y seis acciones disponibles en `Taxi-v4`. Inicialmente todos los valores se establecen a cero.

Durante el entrenamiento, la selección de acción se realiza siguiendo la estrategia ε-greedy. Si un número aleatorio es inferior al valor actual de ε, se selecciona una acción aleatoria. En caso contrario se utiliza:

$$a = \arg\max_a Q(s, a).$$

Una vez ejecutada la acción, se observa la recompensa r y el nuevo estado s'. La actualización Q-Learning puede expresarse mediante:

$$TD_{\text{target}} = r + \gamma \max_{a'} Q(s', a'),$$

$$TD_{\text{error}} = TD_{\text{target}} - Q(s, a),$$

y finalmente:

$$Q(s, a) \leftarrow Q(s, a) + \alpha TD_{\text{error}}.$$

Cuando el episodio termina correctamente, el término correspondiente al valor futuro se considera cero, ya que no existe una recompensa posterior.

En cambio, cuando el episodio se detiene por alcanzar el límite de pasos, se considera una truncación y no una terminación real del problema. En ese caso, el valor estimado del siguiente estado se mantiene en la actualización.

Después de cada episodio se actualiza el nivel de exploración mediante:

$$\varepsilon = \varepsilon_{\min} + (\varepsilon_{\max} - \varepsilon_{\min})e^{-\lambda N},$$

donde N representa el número de episodios completados.

De esta forma, el agente comienza explorando gran parte del entorno y progresivamente pasa a utilizar con mayor frecuencia las acciones aprendidas.

El código correspondiente a la implementación y a los experimentos se entrega por separado.

1.4 Actividad 3 — Cinco experimentos

1.4.1 Experimento 1 — Factor de descuento γ

El primer experimento estudia la influencia del factor de descuento. Comparé los siguientes valores:

$$\gamma \in \{0,\ 0.25,\ 0.5,\ 0.75,\ 1\}.$$

De esta forma se comparan un agente que mira la recompensa immediata ($\gamma = 0$), uno completamente previsor ($\gamma = 1$) y tres valores intermedios distintos de ambos extremos.

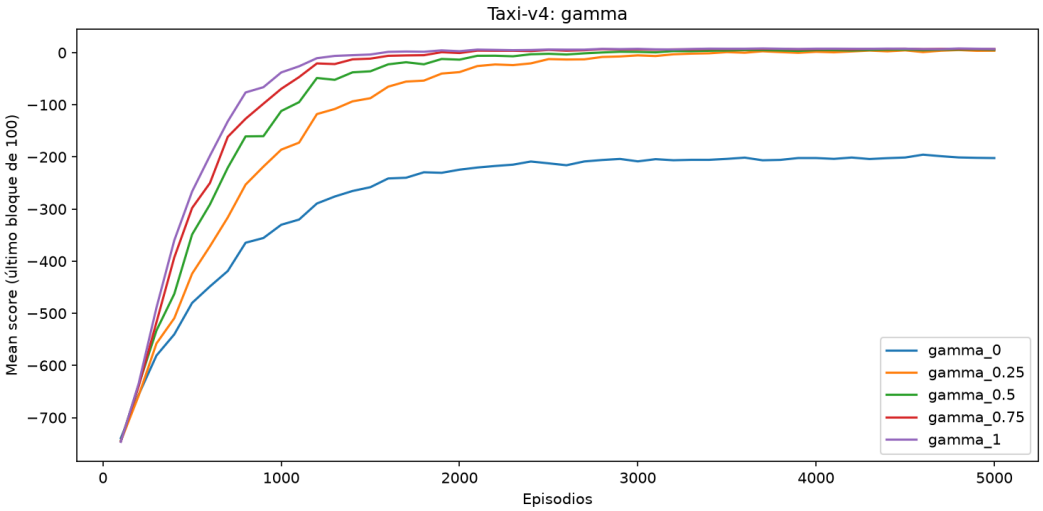


Figure 1: Evolución de la recompensa media para diferentes valores de γ.

Con $\gamma = 0$, el aprendizaje es claramente insuficiente. El último bloque de entrenamiento termina con un

mean score de aproximadamente:

$$-202.15.$$

Al evaluar posteriormente la política sin exploración, ninguno de los 100 episodios se resuelve correctamente. La recompensa media es de -200, correspondiente a alcanzar sistemáticamente el límite de pasos.

El rendimiento mejora progresivamente al aumentar el factor de descuento.

Table 1: Resultados obtenidos para diferentes valores de γ.

γ	**Éxito codicioso**	**Interpretación**
0	0 %	No aprende rutas completas
0.25	49 %	Mejora limitada
0.50	54 %	Mejora moderada
0.75	91 %	Política bastante fiable
1.00	100 %	Mejor resultado

Con $\gamma = 0.75$, el último bloque alcanza un *mean score* de aproximadamente 6.94, mientras que con $\gamma = 1$ aumenta hasta 7.15.

La evaluación codiciosa del agente entrenado con $\gamma = 1$ consigue resolver los 100 episodios, con una recompensa media de 7.86 y aproximadamente 13.14 pasos por episodio.

La explicación principal es que la recompensa por completar la entrega se encuentra diferida. Con factores de descuento bajos, esta recompensa pierde gran parte de su influencia antes de alcanzar los estados iniciales de la trayectoria.

Conclusión. Entre los valores estudiados, se selecciona $\gamma = 1$, ya que proporciona el mejor comportamiento. Este resultado se limita al entorno y a la ejecución realizada y no implica que $\gamma = 1$ sea siempre el mejor valor en cualquier problema de aprendizaje reforzado.

1.4.2 Experimento 2 — Tasa de aprendizaje α/LR

El segundo experimento compara una tasa de aprendizaje baja:

$$\alpha = 0.01,$$

con una tasa considerablemente más elevada:

$$\alpha = 0.7.$$

Ambos agentes se entrenaron durante 5000 episodios.

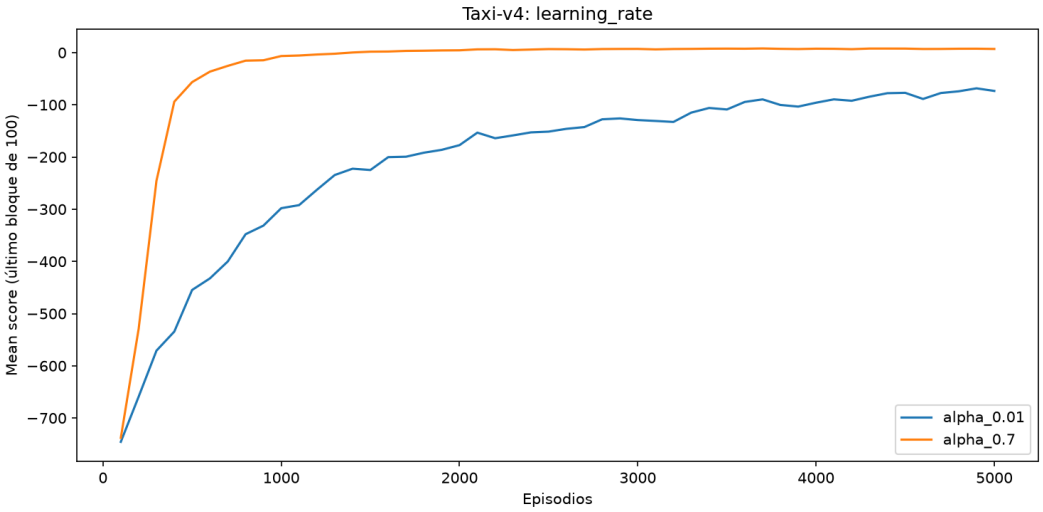


Figure 2: Efecto de la tasa de aprendizaje sobre la recompensa media.

La siguiente tabla resume el resultado final:

Table 2: Comparación de las tasas de aprendizaje.

α	**Mean score final**	**Récord**	**Éxito evaluación**	**Score evaluación**
0.01	-73.47	15	1 %	-197.87
0.70	6.96	15	100 %	7.86

Para analizar con mayor detalle la velocidad de aprendizaje, la Tabla 3 muestra la recompensa media de cada bloque de 100 episodios y el récord acumulado alcanzado hasta ese momento para ambas tasas de aprendizaje.

Table 3: Recompensa media por bloque de 100 episodios y récord acumulado.

	$\alpha = 0.01$		$\alpha = 0.7$	
Episodio	Media	Récord	Media	Récord
100	-745.62	-282	-737.97	-86
200	-659.15	-46	-529.32	-31
300	-570.94	-24	-245.59	5
400	-534.44	-24	-94.04	5
500	-454.67	-19	-56.55	6
600	-432.49	-19	-36.46	6
700	-400.12	-19	-25.46	12
800	-347.91	-19	-15.46	14
900	-331.43	-19	-14.71	15
1000	-298.06	-19	-6.61	15
1100	-292.19	-7	-5.60	15
1200	-262.80	-4	-3.66	15

Table 3: continuación

	$\alpha = 0.01$		$\alpha = 0.7$	
Episodio	Media	Récord	Media	Récord
1300	-234.52	11	-2.20	15
1400	-222.40	11	0.23	15
1500	-224.92	11	1.75	15
1600	-200.27	11	2.03	15
1700	-199.42	11	3.24	15
1800	-191.77	11	3.59	15
1900	-186.29	11	4.14	15
2000	-177.29	11	4.38	15
2100	-153.35	11	6.20	15
2200	-164.04	11	6.36	15
2300	-158.64	11	4.90	15
2400	-152.87	11	5.81	15
2500	-151.37	11	6.72	15
2600	-146.01	11	6.46	15
2700	-142.71	11	5.88	15
2800	-127.78	11	6.79	15
2900	-126.09	14	6.98	15
3000	-129.26	14	7.06	15
3100	-130.94	14	6.13	15
3200	-132.82	14	6.87	15
3300	-114.96	14	7.08	15
3400	-106.09	14	7.46	15
3500	-108.94	15	7.58	15
3600	-94.54	15	7.41	15
3700	-89.60	15	7.89	15
3800	-100.32	15	7.20	15
3900	-103.43	15	6.79	15
4000	-95.98	15	7.44	15
4100	-89.55	15	7.27	15
4200	-92.29	15	6.54	15
4300	-84.43	15	7.71	15
4400	-77.78	15	7.70	15
4500	-77.09	15	7.63	15
4600	-88.82	15	6.90	15
4700	-77.43	15	6.99	15
4800	-74.17	15	7.37	15
4900	-68.47	15	7.43	15
5000	-73.47	15	6.96	15

Ambos entrenamientos alcanzan en algún momento una recompensa máxima de 15. Sin embargo, este valor aislado no permite determinar si el agente ha aprendido una política consistente.

Con $\alpha = 0.01$, el agente aprende muy lentamente. Después de 5000 episodios, la política codiciosa únicamente completa correctamente el 1 % de las evaluaciones.

En cambio, con $\alpha = 0.7$, la información se incorpora con mucha más rapidez y la política final completa correctamente los 100 episodios de evaluación.

Por tanto, el récord máximo de un episodio no es una medida suficiente para comparar agentes. La recompensa media y, especialmente, la evaluación codiciosa permiten analizar mejor la estabilidad de la política aprendida.

Conclusión. Para 5000 episodios, se selecciona $\alpha = 0.7$. Esto no implica que una tasa elevada sea siempre mejor, sino que $\alpha = 0.01$ resulta demasiado lenta en las condiciones de este experimento.

1.4.3 Experimento 3 — Exploración

El tercer experimento analiza dos situaciones extremas en la gestión de la exploración.

En el primer caso se fuerza un abandono prontío de la exploración:

$$\texttt{epsilon_decay_rate} = 0.05.$$

En el segundo se mantiene una exploración permanente:

$$\varepsilon = 0.8.$$

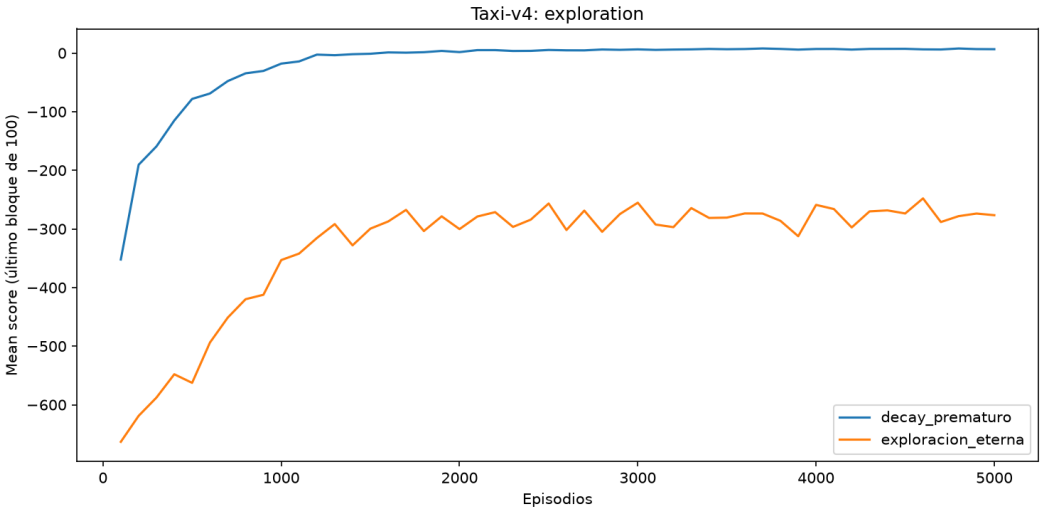


Figure 3: Comparación de estrategias extremas de exploración.

Con poca exploración, la recompensa de entrenamiento mejora rápidamente y el último bloque alcanza aproximadamente:

$$6.56.$$

Sin embargo, al evaluar la política sin exploración únicamente se completan correctamente 82 de los 100 episodios, con una recompensa media de:

$$-28.91.$$

El agente ha explotado demasiado pronto lo que conocía y algunas situaciones iniciales no han sido suficientemente exploradas.

El resultado de la exploración permanente es especialmente interesante. Manteniendo $\varepsilon = 0.8$, el último bloque de entrenamiento obtiene una recompensa media de:

$$-276.64.$$

A primera vista podría parecer que el agente no ha aprendido. Sin embargo, al desactivar la exploración y utilizar exclusivamente la política codiciosa se obtiene:

$$100/100$$

episodios exitosos, con una recompensa media de 7.86.

La diferencia se debe a que durante el entrenamiento el agente sigue tomando acciones aleatorias en el 80 % de las decisiones. La baja recompensa observada refleja principalmente el coste de seguir explorando, no la calidad de la Q-table.

Conclusión. Abandonar la exploración demasiado pronto puede dejar estados insuficientemente aprendidos, mientras que mantenerla permanentemente elevada impide aprovechar la política adquirida. La estrategia más razonable es reducir ε de forma progresiva: explorar al principio y explotar cada vez más conforme aumenta la experiencia.

1.4.4 Experimento 4 — Número de episodios

El cuarto experimento analiza el efecto de aumentar el número de episodios de entrenamiento. Se compararon:

$$500, \quad 2500, \quad 5000, \quad 10000, \quad 20000.$$

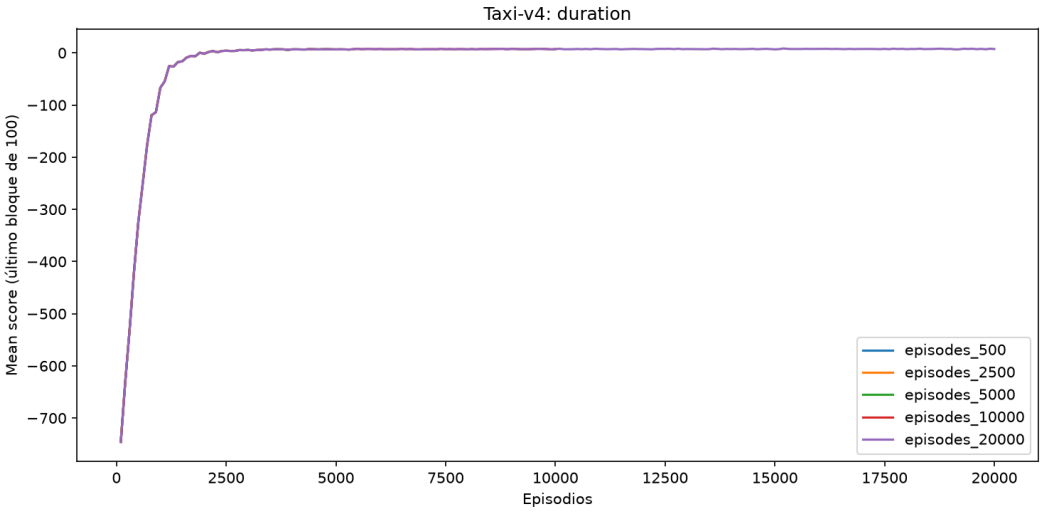


Figure 4: Evolución del aprendizaje según el número de episodios.

Las cinco ejecuciones utilizan la misma configuración y la misma semilla. Por ello, las curvas coinciden exactamente hasta el punto en que termina la ejecución más corta y no deben interpretarse como cinco repeticiones independientes.

Los resultados obtenidos son:

Table 4: Efecto del número de episodios de entrenamiento.

Episodios	**Mean score final**	**Máximo**	**Éxito codicioso**	**Score codicioso**
500	-323.34	2	1 %	-197.87
2500	4.07	15	62 %	-70.18
5000	6.62	15	84 %	-24.99
10000	7.19	15	89 %	-14.64
20000	7.37	15	98 %	3.78

El salto más importante se produce entre 500 y 2500 episodios. La recompensa media del último bloque pasa de -323.34 a 4.07.

A partir de ese punto, las mejoras en recompensa media son cada vez menores. Entre 10000 y 20000 episodios únicamente se pasa de 7.19 a 7.37.

Sin embargo, la tasa de éxito codiciosa continúa aumentando de forma relevante:

$$89\,\% \rightarrow 98\,\%.$$

Esto muestra que una aparente estabilización de la recompensa media no implica necesariamente que la política haya dejado de mejorar.

También es importante observar que el máximo de entrenamiento alcanza 15 ya con 2500 episodios, aunque la

política solamente resuelve correctamente el 62 % de las evaluaciones. Por tanto, un único episodio excelente no indica que el agente haya aprendido una política fiable.

Conclusión. El aprendizaje presenta rendimientos decrecientes a medida que aumenta la experiencia, pero la fiabilidad de la política sigue mejorando incluso cuando la recompensa media parece estabilizarse. Entre las opciones estudiadas, 20000 episodios proporciona el mejor resultado.

1.4.5 Experimento 5 — Agente ideal

A partir de los experimentos anteriores se seleccionaron los hiperparámetros finales.

Para el factor de descuento se utiliza:

$$\gamma = 1,$$

ya que fue el valor con mejor comportamiento en el experimento 1.

Para la tasa de aprendizaje se selecciona:

$$\alpha = 0.7,$$

que consiguió un 100 % de éxito en la evaluación del experimento 2.

Finalmente, se utilizan:

$$20000$$

episodios, ya que fue la duración que produjo el mayor porcentaje de éxito en el experimento 4.

Por tanto, la configuración final es:

$$\boxed{\alpha = 0.7, \qquad \gamma = 1, \qquad N = 20000}$$

Se mantiene el decaimiento gradual de exploración original. Las dos configuraciones del experimento 3 no se utilizaron porque estaban diseñadas precisamente para representar dos situaciones extremas: abandonar la exploración demasiado pronto o mantenerla permanentemente elevada.

El agente final se vuelve a entrenar desde cero, utilizando una nueva Q-table y una semilla diferente de las utilizadas durante la selección de hiperparámetros.

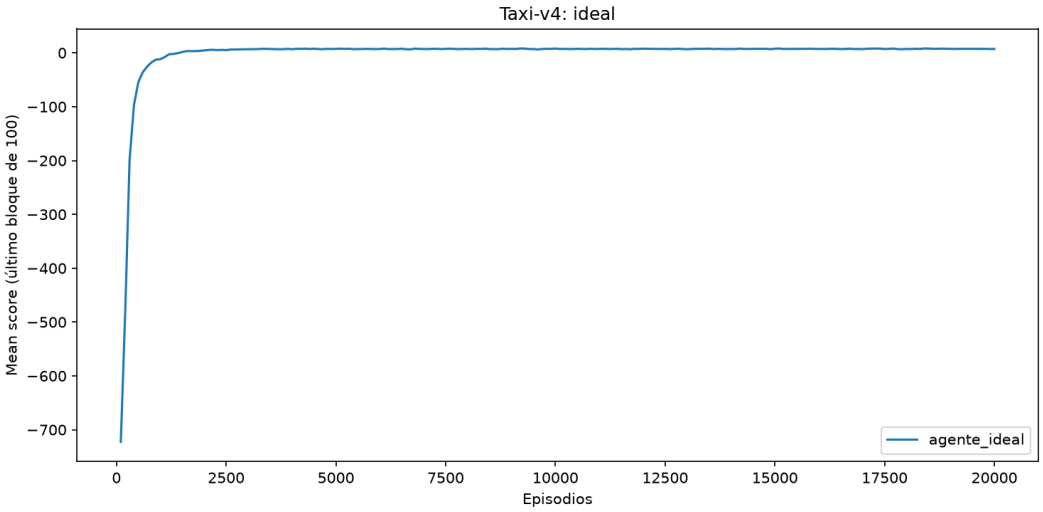


Figure 5: Curva de aprendizaje del agente final.

En el primer bloque de entrenamiento el *mean score* es de aproximadamente:

$$-722.51.$$

En torno al episodio 2500 alcanza:

$$5.01,$$

y alrededor de los 10000 episodios llega a:

$$8.00.$$

El último bloque termina con aproximadamente:

$$7.26.$$

La recompensa máxima observada durante el entrenamiento es 15.

Posteriormente, la política se evalúa de forma codiciosa sobre 100 episodios independientes de los utilizados para seleccionar los hiperparámetros.

El resultado final es:

$$\boxed{100/100 \text{ episodios exitosos}}$$

con una recompensa media de:

7.94

y una duración media de:

13.06 pasos.

Estos resultados muestran que la configuración seleccionada produce una política muy eficaz para las situaciones evaluadas.

Sin embargo, la expresión "agente ideal" debe interpretarse únicamente dentro de los experimentos realizados. Obtener un 100 % de éxito en 100 episodios no demuestra que la política sea perfecta para todos los posibles estados o para variantes diferentes del entorno.

Además, cada configuración se ha entrenado utilizando una única semilla. Una mejora metodológica sería repetir cada experimento con varias semillas y comparar la distribución de resultados, permitiendo estimar mejor la estabilidad de los hiperparámetros seleccionados.

También sería interesante estudiar específicamente aquellos estados iniciales que producen trayectorias más largas y evaluar estrategias de exploración dirigidas a las zonas menos visitadas del espacio de estados.

1.5 Actividad 4 — SlimeVolleyGym

1.5.1 Pregunta 1 — Espacios de estado

En `Taxi-v4`, el estado se representa mediante un único entero entre 0 y 499. Esto permite utilizar directamente el estado como índice de una fila de la Q-table.

En cambio, `SlimeVolley-v0` representa el estado mediante un vector continuo de 12 valores que contiene información relacionada con el agente, el rival y la pelota.

Por tanto, la principal diferencia es que Taxi presenta un espacio discreto y reducido, mientras que SlimeVolley utiliza un espacio continuo y multidimensional.

1.5.2 Pregunta 2 — Límites de una Q-table

Una Q-table necesita almacenar explícitamente un valor para cada combinación posible de estado y acción.

Esto resulta viable en Taxi porque solo existen 500 estados.

En SlimeVolley, cada componente del vector puede adoptar una gran cantidad de valores posibles. Enumerar todas las combinaciones de 12 dimensiones generaría un número extremadamente elevado de estados.

Incluso realizando una discretización de las variables, el tamaño de la tabla crecería rápidamente y gran parte de sus estados probablemente nunca serían visitados durante el entrenamiento.

Además, dos estados numéricamente muy parecidos se tratarían como entradas completamente diferentes si no coincidieran exactamente en el mismo índice.

Por tanto, una Q-table convencional no resulta una representación adecuada para este tipo de espacio continuo.

1.5.3 Pregunta 3 — Uso de redes neuronales

Una red neuronal permite aproximar una función mediante un número finito de parámetros:

$$f(s;\theta),$$

donde s representa el estado y θ los parámetros de la red.

En lugar de almacenar explícitamente una fila diferente para cada estado, la red aprende relaciones generales entre las variables de entrada y las decisiones.

La principal ventaja es que puede generalizar entre estados similares. Una situación ligeramente diferente de otra ya observada puede generar una respuesta parecida aunque el agente nunca haya visto exactamente ese mismo vector.

Esto permite trabajar con espacios continuos de una manera mucho más eficiente que mediante una tabla explícita.

1.5.4 Pregunta 4 — Representación de la red

En Taxi, la Q-table representa de forma explícita:

$$Q(s,a).$$

Una red neuronal puede reemplazar esta representación.

En un enfoque basado en valores Q, la red podría recibir como entrada el vector de estado y producir como salida los valores estimados de las diferentes acciones:

$$Q(s,a_1),\ Q(s,a_2),\ \ldots$$

En algoritmos basados directamente en políticas, como PPO, la red puede producir probabilidades asociadas a las acciones disponibles.

Por tanto, la red sustituye la tabla explícita por una función parametrizada capaz de producir decisiones para un número muy grande de estados diferentes.

1.5.5 Pregunta 5 — Ventaja de las redes neuronales

Una ventaja concreta es la capacidad de generalización.

Supongamos que durante el entrenamiento el agente observa una situación en la que la pelota se encuentra en una determinada posición. Posteriormente aparece prácticamente la misma situación, pero la pelota está ligeramente desplazada.

Una Q-table necesitaría disponer de una entrada específica para ese nuevo estado.

Una red neuronal, en cambio, puede reutilizar los patrones aprendidos anteriormente y producir una respuesta similar.

Esta capacidad resulta especialmente importante en SlimeVolley, donde los estados son continuos y es prácticamente imposible visitar exactamente todas las combinaciones posibles.

1.6 Conclusiones

Los experimentos permiten observar de forma práctica cómo los principales hiperparámetros de Q-Learning afectan al comportamiento del agente.

El factor de descuento γ resulta especialmente importante en `Taxi-v4`. Con $\gamma = 0$, el agente no consigue resolver ninguno de los 100 episodios de evaluación. Al aumentar progresivamente el descuento, el rendimiento mejora hasta alcanzar un 100 % de éxito con $\gamma = 1$. Esto confirma que, en este entorno, resulta fundamental valorar las consecuencias futuras de las acciones.

La tasa de aprendizaje también tiene un efecto considerable. Con 5000 episodios, $\alpha = 0.01$ permite únicamente un 1 % de éxito en evaluación, mientras que $\alpha = 0.7$ alcanza el 100 %. En estas condiciones, la tasa baja incorpora la información demasiado lentamente.

El experimento de exploración muestra además que la recompensa obtenida durante entrenamiento no debe interpretarse directamente como la calidad de la política aprendida. El agente que mantiene $\varepsilon = 0.8$ obtiene un resultado de entrenamiento muy negativo, pero su política codiciosa posterior resuelve los 100 episodios. La exploración sigue perjudicando el rendimiento observado aunque la Q-table ya contenga buenas acciones.

El análisis del número de episodios confirma que alcanzar una recompensa máxima elevada tampoco implica haber aprendido una política robusta. Con 2500 episodios ya se consigue alguna vez la recompensa 15, pero únicamente se resuelve correctamente el 62 % de las evaluaciones.

Al aumentar el entrenamiento hasta 20000 episodios, la política alcanza un 98 % de éxito en el experimento de duración. Aunque la recompensa media de entrenamiento muestra rendimientos decrecientes, la fiabilidad del agente sigue aumentando.

A partir de estos resultados se seleccionó la configuración:

$$\boxed{\alpha = 0.7, \qquad \gamma = 1, \qquad N = 20000}$$

y se entrenó una nueva Q-table desde cero.

La evaluación final consigue:

$$100/100$$

episodios exitosos, con una recompensa media de 7.94 y aproximadamente 13.06 pasos por episodio.

En consecuencia, Q-Learning resulta una solución muy efectiva para `Taxi-v4`, donde el espacio de estados es discreto y suficientemente pequeño como para representar explícitamente los valores mediante una Q-table.

Sin embargo, esta representación deja de ser práctica cuando el espacio de estados pasa a ser continuo y de alta dimensión, como ocurre en SlimeVolley. En ese tipo de problemas es necesario utilizar aproximadores de funciones, como redes neuronales, capaces de generalizar entre estados similares.

Finalmente, los resultados experimentales deben interpretarse teniendo en cuenta que cada configuración se entrenó con una única semilla. Para obtener conclusiones más sólidas sobre la estabilidad de los hiperparámetros sería conveniente repetir los experimentos con varias semillas y comparar la distribución de los resultados.

1.7 Bonus: DCCampeonato.

Para el campeonato entrené un agente Q-Learning desde una Q-table vacía durante el máximo permitido de 650 episodios. Elegí una tasa de aprendizaje $\alpha = 0{,}8$, un factor de descuento $\gamma = 0{,}99$ y una política ε-greedy con exploración inicial de 1, mínima de 0,05 y tasa de decaimiento de 0,005. El descuento alto favorece aprender rutas que conducen a completar el viaje; el decaimiento permite que el agente explore al principio y aproveche lo aprendido dentro del presupuesto limitado. La función `DCCampeonato()` genera `q_table_campeonato.csv` con 500 filas y siete columnas, sin cabecera. Al evaluar la política obtenida sin exploración en 200 episodios con semillas distintas de las de entrenamiento, completó 187 viajes (93,5 %). Este resultado corresponde a una evaluación local y no garantiza el mismo rendimiento frente a los casos utilizados en el torneo.

References

[1] C. J. C. H. Watkins y P. Dayan, "Q-learning," *Machine Learning*, vol. 8, pp. 279–292, 1992. https://doi.org/10.1007/BF00992698

[2] Farama Foundation, "Taxi-v4," documentación de Gymnasium. https://gymnasium.farama.org/environments/toy_text/taxi/

[3] Farama Foundation, "Handling Time Limits," documentación de Gymnasium. https://gymnasium.farama.org/main/tutorials/handling_time_limits/

[4] D. Ha, "Slime Volleyball Gym Environment," repositorio de GitHub, 2020. https://github.com/hardmaru/slimevolleygym

[5] IIC2613 Inteligencia Artificial, *Tarea 1: Aprendizaje Automático y Reforzado*, Pontificia Universidad Católica de Chile, 2026-2. Material proporcionado por el curso.

1.8 Anexo de IA

Para la realización de esta parte se utilizaron herramientas de inteligencia artificial generativa como apoyo en distintas fases del trabajo.

En particular, se utilizaron para:

- comprender y revisar el funcionamiento del algoritmo Q-Learning;
- revisar la lógica de actualización de la Q-table;
- estructurar los experimentos requeridos;
- interpretar los resultados obtenidos durante las ejecuciones;
- organizar y revisar la redacción final del informe.

Los resultados numéricos incluidos en el documento proceden de las ejecuciones realizadas con el código entregado y no de la herramienta de inteligencia artificial.

- **Herramientas de IA utilizadas:** se utilizó Chatgpt como herramienta de apoyo para la generación y revisión del código correspondiente a esta parte del proyecto.
- **Finalidad de su uso:** se utilizó principalmente para agilizar la implementación de los experimentos, la construcción del *pipeline* de entrenamiento y evaluación, así como la generación automática de gráficos, tablas y archivos de resultados que posteriormente fueron analizados e interpretados. También se empleó como apoyo en la preparación de las referencias utilizadas en el informe.
- **Prompts relevantes:**
  - 'Ayúdame a desarrollar la parte obligatoria y el bonus DCCampeonato: 1. Implementa Q-Learning tabular en `TaxiAgent.py`: Q-table de 500 estados por 6 acciones, política ε-greedy, actualización Q y decaimiento de la exploración. Usa `train.py` para entrenar y registrar resultados. Conserva `main.py` y `test.py` sin modificar, como exige el enunciado.

    2. Prepara evidencia reproducible para los cinco experimentos obligatorios: comparación de $\gamma = 0$ y $\gamma = 1$ más tres valores intermedios; comparación de tasas de aprendizaje baja y alta con métricas cada 100 episodios; exploración prematura y permanente; entrenamientos de 500, 2500, 5000, 10000 y 20000 episodios; y un agente final justificado. Respeta el mínimo de 5000 episodios donde corresponda.

    No inventes métricas ni afirmes que algo funciona sin ejecutarlo. Guarda las configuraciones, semillas, tablas y gráficos necesarios para que yo pueda revisar los resultados. Distingue claramente entre hipótesis, resultados medidos e interpretaciones. Antes de dar el trabajo por terminado, comprueba el límite de episodios, el formato del CSV y que los archivos originales exigidos permanecen sin cambios."
  - *"Implementa `DCCampeonato()` en `train.py` con un máximo de 650 episodios."*
  - *"Ayúdame a preparar un borrador de las respuestas teóricas de las actividades 1 y 4 y a interpretar los experimentos. Cita fuentes oficiales o primarias para las afirmaciones externas."*
- **Respuestas relevantes:** la respuesta principal utilizada fue el código generado y posteriormente ejecutado para esta parte del proyecto, junto con la estructura de experimentos, tablas, gráficos y archivos de resultados obtenidos. Estos resultados fueron revisados posteriormente y utilizados como base para realizar el análisis presentado en el informe.

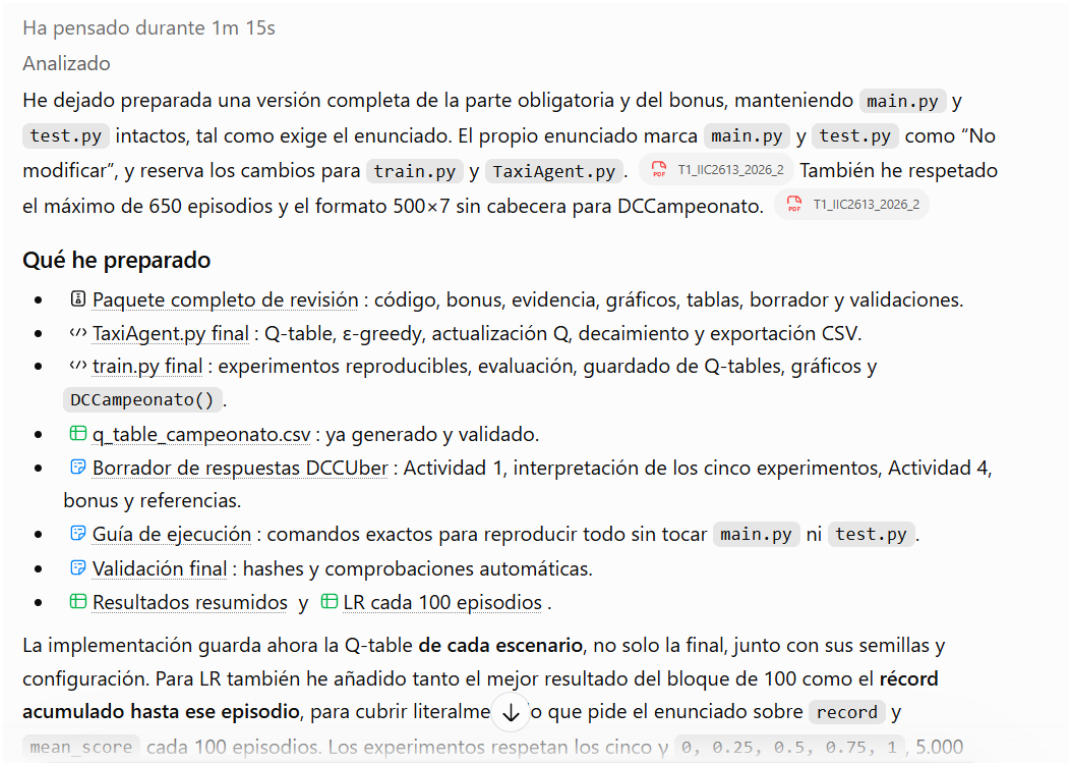


Figure 6: Respuesta de ChatGPT